МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №4

по дисциплине: **Информационные элементы робототехнических систем**

Тема лабораторной работы: «Обучение модели для предсказания фруктов на изображении»

Студенты группы: КРБО-03-23

Зенина А.А. _____

Грачев А.В. _____

Гришаев А.К. _____

Преподаватель: ст. преподаватель Бычков А.М. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Переключаемся на cuda

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
import os

# Принудительно ставим GPU (даже если проверка виснет)
device = torch.device('cuda')   # ← напрямую указываем cuda

print(f"Принудительно используем: {device}")

#
```

```
Принудительно используем: cuda
```

Переключение на видюху чтобы лучше работало

```
import torch

print("CUDA версия в PyTorch:", torch.version.cuda)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

else:
    print(" GPU не работает")
```

```
CUDA версия в PyTorch: 12.6
GPU: NVIDIA GeForce RTX 4060 Laptop GPU
```

Пути для папок с выборками, Размеры батчей, колличество эпох, скорость обучения, параметры для ранней остановки если нету улучшений

```
train_path = r"C:\Users\admin_local\.cache\kagglehub\datasets\
youssefsalahzakria\fruit-and-vegetables-classification\versions\5\
train"
val_path   = r"C:\Users\admin_local\.cache\kagglehub\datasets\
youssefsalahzakria\fruit-and-vegetables-classification\versions\5\
validation"
test_path  = r"C:\Users\admin_local\.cache\kagglehub\datasets\
youssefsalahzakria\fruit-and-vegetables-classification\versions\5\
test"
```

```
batch_size = 32
num_epochs = 10
learning_rate = 0.001

# Параметры для Early Stopping
patience = 5
min_delta = 0.001
```

Преобразование изображений для обработки. Случайным образом уменьшаем/увеличиваем, поворачиваем на n градусов регулируем цвета и тд.

```
# Улучшенные трансформации
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.4, contrast=0.4,
saturation=0.4),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

test_transform = val_transform  # для теста используем те же, что и
для валидации


train_dataset = datasets.ImageFolder(root=train_path,
transform=train_transform)
val_dataset   = datasets.ImageFolder(root=val_path,
transform=val_transform)
test_dataset  = datasets.ImageFolder(root=test_path,
transform=test_transform)


classes = train_dataset.classes
num_classes = len(classes)
```

```
print(f"Найдено классов: {num_classes}")
print(f"Классы: {classes}")
print(f"Размер train: {len(train_dataset)} изображений")
print(f"Размер val:   {len(val_dataset)} изображений")
print(f"Размер test:  {len(test_dataset)} изображений")
```

```
Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__
at 0x000002304FEFF7E0>
Traceback (most recent call last):
  File "c:\Users\admin_local\AppData\Local\Programs\Python\Python313\
Lib\site-packages\torch\utils\data\dataloader.py", line 1709, in
__del__
    self._shutdown_workers()
  File "c:\Users\admin_local\AppData\Local\Programs\Python\Python313\
Lib\site-packages\torch\utils\data\dataloader.py", line 1667, in
_shutdown_workers
    if self._persistent_workers or self._workers_status[worker_id]:
AttributeError: '_MultiProcessingDataLoaderIter' object has no
attribute '_workers_status'

Найдено классов: 50
Классы: ['Apple', 'Avocado', 'Banana', 'Beetroot', 'Blackberry',
'Blueberry', 'Broccoli', 'Cabbage', 'Capsicum', 'Carrot',
'Cauliflower', 'Chilli Peper', 'Corn', 'Cucumber', 'Dates',
'Dragonfruit', 'Eggplant', 'Fig', 'Garlic', 'Ginger', 'Grapes',
'Guava', 'Jalepeno', 'Kiwi', 'Lemon', 'Lettuce', 'Mango', 'Mushroom',
'Okra', 'Olive', 'Onion', 'Orange', 'Paprika', 'Peanuts', 'Pear',
'Peas', 'Pineapple', 'Pomegranate', 'Potato', 'Pumpkin', 'Raddish',
'Rambutan', 'Soy Beans', 'Spinach', 'Strawberry', 'Sweetcorn',
'Sweetpotato', 'Tomato', 'Turnip', 'Watermelon']
Размер train: 46187 изображений
Размер val:   12544 изображений
Размер test:  13353 изображений
```

Создание датасетов для обработки изображений (для трейн всегда изображения в разном порядке) num_workers-сколько процессов загружают данные (статичные батчи для тренировки ухудшают сильно обучение до точности в 1 процент, а если валидационные мешать, то оно тоже падает, но не сильно)

```
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=4,
    pin_memory=True
)

val_loader = DataLoader(
```

```
    val_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=4,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=4,
    pin_memory=True
)

print(f"Train batches: {len(train_loader)}")
print(f"Val batches:   {len(val_loader)}")
print(f"Test batches:  {len(test_loader)}")
```

```
Train batches: 1444
Val batches:   392
Test batches:  418
```

Структура модели тута построена почти так же как и в лекции (но больше циклов чем там)

Feature extractor (свёрточная часть): Conv2d 3→64 → BatchNorm → ReLU → MaxPool→ Conv2d 64→128 → BatchNorm → ReLU → MaxPool→ Conv2d 128→256 → BatchNorm → ReLU → MaxPool→ Conv2d 256→512 → BatchNorm → ReLU → MaxPool

Classifier (классифицирующая часть): AdaptiveAvgPool2d(1,1) — превращает карту признаков в один вектор Dropout(0.5) Linear(512 → 512) → ReLU → Dropout(0.5)→ Linear(512 → 50) — 50 классов

```
class ImprovedCNN_Light(nn.Module):
    def __init__(self, num_classes):
        super().__init__()

        self.features = nn.Sequential(
            # 3 → 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),  # 112x112

            # 32 → 64
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),  # 56x56
```

```
            # 64 → 128
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),  # 28x28

            # 128 → 256 (вместо 512)
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),  # 14x14
        )

        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Dropout(0.5),
            nn.Linear(256, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

model = ImprovedCNN_Light(num_classes).to(device)
print(f"Параметров: {sum(p.numel() for p in model.parameters()):,}")
```

```
Параметров: 468,018
```

потери, оптимизатор и борьба с переобучением

```
criterion = nn.CrossEntropyLoss()

optimizer = optim.AdamW(
    model.parameters(),
    lr=learning_rate,
    weight_decay=1e-4
)

\
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=3,
```

```
)
```

Сначала модель учится на тренировочных данных: она проходит по всем примерам маленькими порциями, предсказывает ответы, смотрит, сколько ошиблась, и постепенно подкручивает свои настройки, чтобы ошибаться меньше. После этого её проверяем на отдельном валидационном наборе — там она уже не учится, а просто считает свои ошибки и точность, чтобы мы видели, как хорошо она работает на новых данных.

Если точность на проверке оказалась лучше, чем раньше, то лучшую версию модели сохраняют в файл. А если точность не улучшается пять эпох подряд, то обучение останавливают досрочно, чтобы не тратить время.

```
# ================================================
# БЛОК 7: Обучение с валидацией и Scheduler
# ================================================

train_losses = []
val_losses = []
val_accuracies = []

best_val_loss = float('inf')
best_val_accuracy = 0.0
patience_counter = 0
best_model_path = "best_model.pth"

model.train()

for epoch in range(num_epochs):
    # ==================== TRAINING ====================
    model.train()
    train_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()

    avg_train_loss = train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # ==================== VALIDATION ====================
```

```
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    avg_val_loss = val_loss / len(val_loader)
    val_accuracy = 100 * correct / total

    val_losses.append(avg_val_loss)
    val_accuracies.append(val_accuracy)

    # Обновляем scheduler
    scheduler.step(avg_val_loss)

    # Вывод информации
    print(f"Эпоха [{epoch+1}/{num_epochs}]  "
          f"Train Loss: {avg_train_loss:.4f}  |  "
          f"Val Loss: {avg_val_loss:.4f}  |  "
          f"Val Acc: {val_accuracy:.2f}%")

    # Сохранение лучшей модели (по валидационной точности)
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        patience_counter = 0
        torch.save(model.state_dict(), best_model_path)
        print(f"   → Сохранена новая лучшая модель! Val Acc:
{val_accuracy:.2f}%")
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"Early Stopping! Нет улучшения {patience} эпох
подряд.")
            break

print("\nОбучение завершено!")
print(f"Лучшая модель сохранена в: {best_model_path}")
print(f"Лучшая валидационная точность: {best_val_accuracy:.2f}%")
```

```
Эпоха [1/10]  Train Loss: 3.0392  |  Val Loss: 2.6039  |  Val Acc:
25.92%
   → Сохранена новая лучшая модель! Val Acc: 25.92%
Эпоха [2/10]  Train Loss: 2.7114  |  Val Loss: 2.3402  |  Val Acc:
33.28%
   → Сохранена новая лучшая модель! Val Acc: 33.28%
Эпоха [3/10]  Train Loss: 2.5463  |  Val Loss: 2.2518  |  Val Acc:
37.13%
   → Сохранена новая лучшая модель! Val Acc: 37.13%
Эпоха [4/10]  Train Loss: 2.4130  |  Val Loss: 2.1022  |  Val Acc:
43.98%
   → Сохранена новая лучшая модель! Val Acc: 43.98%
Эпоха [5/10]  Train Loss: 2.2979  |  Val Loss: 1.9957  |  Val Acc:
45.14%
   → Сохранена новая лучшая модель! Val Acc: 45.14%
Эпоха [6/10]  Train Loss: 2.1858  |  Val Loss: 1.8007  |  Val Acc:
49.82%
   → Сохранена новая лучшая модель! Val Acc: 49.82%
Эпоха [7/10]  Train Loss: 2.0861  |  Val Loss: 1.7695  |  Val Acc:
51.13%
   → Сохранена новая лучшая модель! Val Acc: 51.13%
Эпоха [8/10]  Train Loss: 1.9984  |  Val Loss: 1.6273  |  Val Acc:
55.52%
   → Сохранена новая лучшая модель! Val Acc: 55.52%
Эпоха [9/10]  Train Loss: 1.9079  |  Val Loss: 1.5820  |  Val Acc:
56.02%
   → Сохранена новая лучшая модель! Val Acc: 56.02%
Эпоха [10/10]  Train Loss: 1.8373  |  Val Loss: 1.4799  |  Val Acc:
58.69%
   → Сохранена новая лучшая модель! Val Acc: 58.69%

Обучение завершено!
Лучшая модель сохранена в: best_model.pth
Лучшая валидационная точность: 58.69%
```

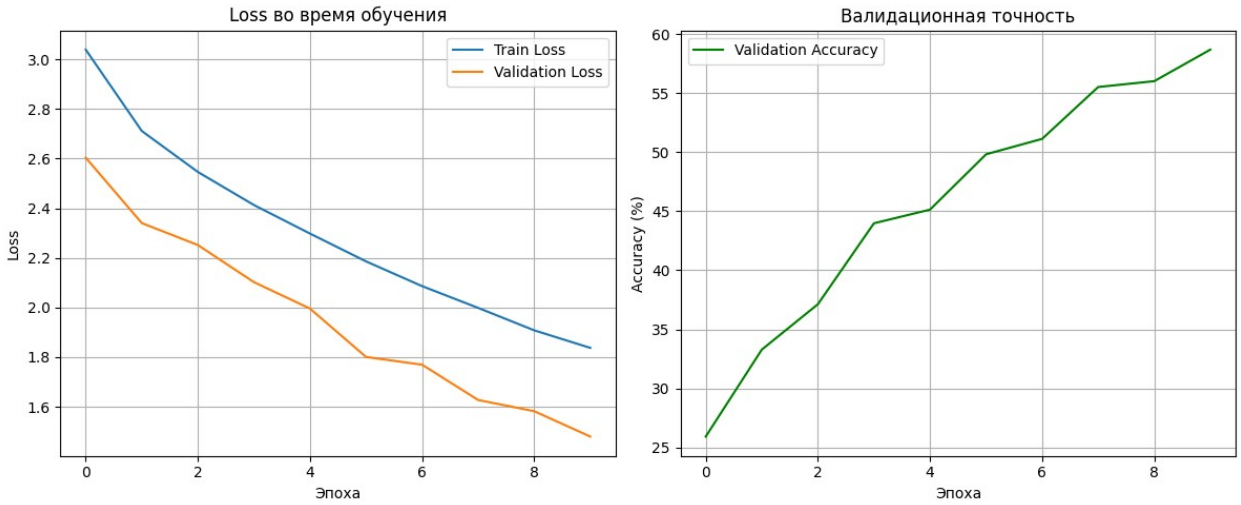
пару графиков для наглядности

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.title('Loss во время обучения')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
```

```
plt.subplot(1, 2, 2)
plt.plot(val_accuracies, label='Validation Accuracy', color='green')
plt.title('Валидационная точность')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Тестируем лучшую модель третьей тестовой выборкой

```
# Загружаем лучшую модель
model.load_state_dict(torch.load(best_model_path))
model.eval()

correct = 0
total = 0
test_loss = 0.0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        test_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

```
test_accuracy = 100 * correct / total
avg_test_loss = test_loss / len(test_loader)

print(f"Test Loss: {avg_test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.2f}%")

Test Loss: 1.6716
Test Accuracy: 53.55%
```

Проверка на ихображении не из датасетов

```
import torch
from PIL import Image
from torchvision import transforms
import matplotlib.pyplot as plt

image_path = r"C:\Users\admin_local\Documents\AIR\AI_metod\
7a9161c680da65f68f5070d64b10689a.jpg"

 # или .png, .jpeg

transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])


model.eval()   # переводим модель в режим оценки


try:
    image = Image.open(image_path).convert('RGB')
except FileNotFoundError:

    raise

input_tensor = transform(image).unsqueeze(0).to(device)   # добавляем
batch dimension


with torch.no_grad():
    output = model(input_tensor)
    probabilities = torch.nn.functional.softmax(output[0], dim=0)

    # Получаем лучший класс и уверенность
    pred_idx = torch.argmax(probabilities).item()
    confidence = probabilities[pred_idx].item() * 100
```

```
predicted_class = classes[pred_idx]




print(f"Предсказанный класс: **{predicted_class}**")
print(f"Уверенность: {confidence:.2f}%")




plt.figure(figsize=(6, 6))
plt.imshow(image)
plt.title(f"Предсказание: {predicted_class}\nУверенность:
{confidence:.1f}%",
          fontsize=14, fontweight='bold')
plt.axis('off')
plt.show()
```

```
Предсказанный класс: **Pomegranate**
Уверенность: 76.23%
```

Предсказание: Pomegranate
Уверенность: 76.2%